Estudo Dirigido — TAD Pilha

Algoritmos e Estrutura de Dados — PUC Minas, Campus Contagem — 2026/2

Integrantes (nome completo e matrícula)

Nome completo	Matrícula

Parte 2 — Rastreio no papel (começando com new Pilha(4))

Operação	Retorno	Pilha (base → topo)	topo
empilha(10)	—	[10]	1
empilha(20)	—	[10, 20]	2
desempilha()	20	[10]	1
empilha(30)	—	[10, 30]	2
empilha(40)	—	[10, 30, 40]	3
desempilha()	40	[10, 30]	2
desempilha()	30	[10]	1
empilha(50)	—	[10, 50]	2

Pare e pense. Ao final, **item[2] guarda o valor 40** (foi escrito por empilha(40) e nunca sobrescrito). Esse valor **não faz mais parte da pilha**: com topo = 2, só as posições 0 e 1 são válidas. desempilha não precisa apagar nada porque a pilha é definida pelo valor de topo, e não pelo conteúdo do vetor: basta decrementar topo, e a posição passa a ser “livre”. O resíduo fica no vetor, mas nenhuma operação do TAD consegue lê-lo (consultaTopo e desempilha só olham item[topo − 1]), e o próximo empilha o sobrescreve. Apagar seria trabalho extra sem efeito algum, e assim desempilha continua em O(1).

Parte 4.1 — Rastreio do Bubble Sort com pilhas, vetor {5, 2, 9, 1, 7}

A começa com 5 na base e 7 no topo. O estado abaixo é o do final de cada passada, antes de trocar os papéis de A e B.

Passada	carry final	B (base → topo)	R (base → topo)
1	9	[1, 7, 2, 5]	[9]
2	7	[2, 5, 1]	[9, 7]
3	5	[1, 2]	[9, 7, 5]
4	2	[1]	[9, 7, 5, 2]
5	1	[ ] (vazia)	[9, 7, 5, 2, 1]

Ao desempilhar R obtém-se 1, 2, 5, 7, 9 (ordem crescente).

Parte 4.3 — Questões

(a) Comparações. Para n = 5 foram feitas **10 comparações** (4 + 3 + 2 + 1 + 0). Em geral, a passada com restantes elementos faz restantes − 1 comparações, então o total é (n − 1) +

$(n-2) + \ldots + 1 = \mathbf{n(n-1)/2}$. O Bubble Sort clássico sem otimização faz exatamente o mesmo número, $n(n-1)/2$, pois também percorre todos os pares das passadas; a versão com pilhas não ganha nem perde em comparações, e esse total não depende dos dados (um vetor já ordenado também gasta 10 comparações para $n = 5$).

(b) Custo extra de memória. O Bubble Sort com vetor ordena no próprio vetor e usa memória extra $O(1)$ (uma variável temporária para a troca). Com pilhas, são alocadas três pilhas de capacidade n (A, B e R, ou seja, $3n$ posições) e ainda o vetor de saída com n posições, então o custo extra é **$O(n)$**. Em cada instante há só n elementos distribuídos entre as pilhas, mas a capacidade alocada é $3n$.

(c) Operação perdida. Deixamos de ter o **acesso direto por índice** (e, com ele, a troca de duas posições v[j] e v[j+1]): na pilha só se enxerga o topo. O código compensa mantendo a "bolha" na variável carry, que faz o papel do elemento que andaria pelo vetor, e usando a pilha B para guardar o que já foi visto na passada. Como passar de A para B inverte a ordem dos elementos, as pilhas A e B trocam de papel a cada passada, e o maior dos restantes é empilhado em R.

(d) Opinião. Em geral **não vale a pena**: a quantidade de comparações é a mesma do Bubble Sort com vetor ($O(n^2)$), mas gasta mais memória ($O(n)$ extra) e faz muito mais operações de empilha/desempilha. Eu usaria essa ideia quando só fosse possível acessar os dados em ordem LIFO, como ao ordenar uma própria pilha (o desafio), ou ao processar dados que chegam e são consumidos só pelo topo. Fora isso, é útil sobretudo como exercício para entender o que o TAD permite e o que não permite.